EBS Snapshots and Backup

Explore how to create and manage Amazon EBS snapshots to efficiently back up block storage. Learn best practices for snapshot automation using AWS Data Lifecycle Manager, cross-region replication for disaster recovery, and integration into CI/CD workflows. Understand how snapshots ensure application resilience and data recovery with minimal disruption.

An Amazon EBS snapshot is a point-in-time backup of the data stored on an EBS volume. It captures the volume's state at the moment the snapshot is initiated and is stored in Amazon S3, which is managed transparently by AWS. Snapshots are the foundational mechanism for backing up block storage on EC2 instances and are commonly used for compliance, testing, migration, and recovery.

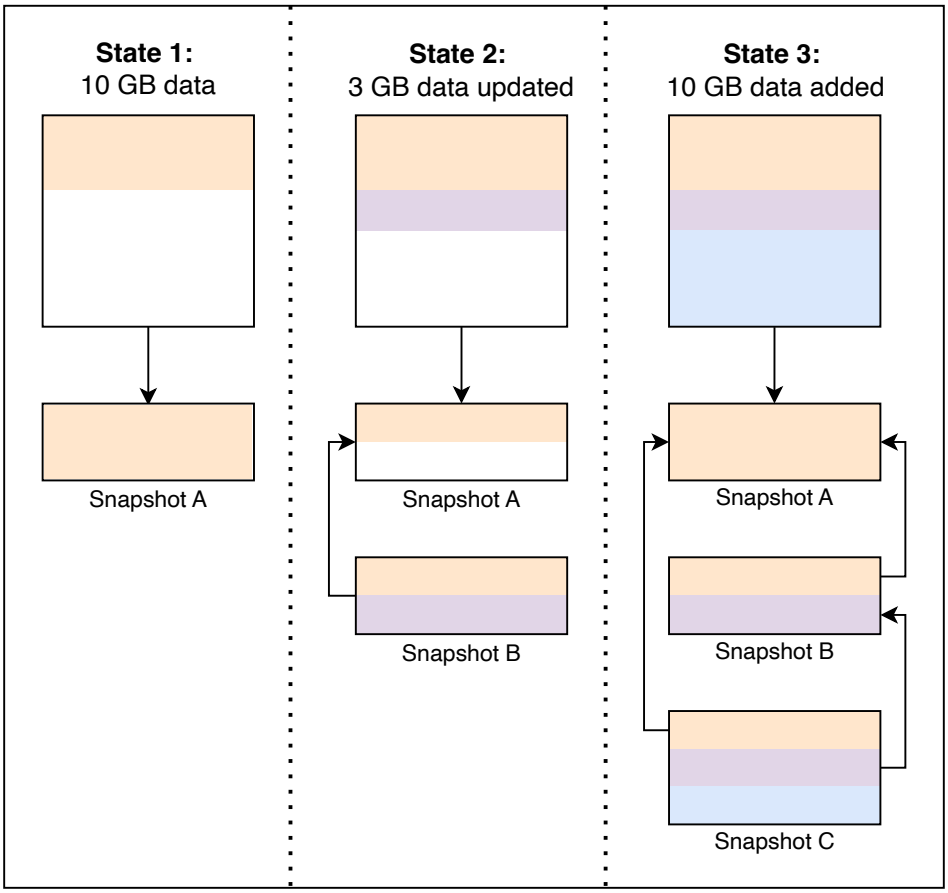
How EBS snapshots work

EBS snapshots are a low-level backup mechanism that creates a point-in-time copy of the data stored on an EBS volume. These snapshots are incremental and highly optimized for performance and cost-efficiency, which is why they are the preferred method for data protection at the block level.

When we take our first snapshot, it is a complete copy of all the data blocks on the volume. However, every subsequent snapshot only captures blocks that have changed since the previous snapshot. This block-level change tracking ensures efficient storage usage, reducing both time and cost.

Ask

> **Note:** To restore a volume, we can use any snapshot independently, as AWS automatically assembles the full volume view using the chain of snapshots.



EBS snapshots

Snapshots can be taken from volumes that are in use, such as those attached to running EC2 instances. In these cases, AWS uses I/O quiescing mechanisms to flush pending writes and ensure snapshot consistency. On Linux systems, `fsfreeze` can be used to pause file system activity. On Windows, AWS supports the Volume Shadow Copy Service (VSS).

Ask

> **Best practice:** Use `aws ec2 create-snapshot --volume-id <vol-id> --description "Backup"` alongside I/O quiescing tools to ensure a

> consistent backup of running applications, especially databases.

Creating snapshots

EBS snapshots can be created manually whenever a consistent backup of an Amazon EBS volume is needed. This is especially useful before performing system updates, configuration changes, or deploying new application versions.

We can create a snapshot using the following AWS CLI command. The AWS CLI provides a scriptable and repeatable way to create snapshots. It is especially valuable in automation workflows:

main.sh Shell

```
aws ec2 create-snapshot \
--volume-id <Volume_ID> \
--description "Manual backup before update" \
--tag-specifications 'ResourceType=snapshot,Tags=[{Key=Name,Value=Sam
```

Create a snapshot using AWS CLI

We can also use AWS SDKs to create snapshots. The following code snippet uses Python and `boto3` to create a snapshot of the EBS volume and add a name tag to it for identification at later stages:

Ask

main.py Python

```
import boto3

ec2 = boto3.client('ec2')

# Step 1: Create the snapshot
response = ec2.create_snapshot(
    VolumeId='<Volume_ID>',
    Description='Manual snapshot via script'
)

snapshot_id = response['SnapshotId']
print("Snapshot ID:", snapshot_id)
```

Create EBS snapshot using Python

Using scripts to generate snapshots is ideal for use cases where we need to integrate the snapshot creation into CI/CD pipelines.

> **Best practice:** Always label snapshots with meaningful tags (e.g., `Environment=Prod`, `BackupType=PreUpdate`) for easier management and filtering later.

Snapshots can also be created through the AWS Management Console by selecting the volume and choosing “Create Snapshot” from the Actions menu. However, for infrastructure-as-code practices and automation, using the CLI or SDK is recommended.

This method is ideal for users who prefer a graphical interface or are managing a small number of volumes.

Snapshot cross-Region resilience

EBS snapshots are created in a single AWS Region by default, but they ca copied to other regions for disaster recovery (DR), compliance, and resilience. This allows us to recover from outages affecting an entire region.

Ask

Storing snapshots across regions

Cross-Region copies are fully functional snapshots and can be used to launch new volumes in the destination region. This approach decouples our backups from the original region's infrastructure.

index.js C++

```
aws ec2 copy-snapshot \
  --source-region us-east-1 \
  --source-snapshot-id <valid_snapshot_id> \
  --region us-west-1 \
  --description "Cross-region backup"
```

Ask

Cross-Region copy of an Amazon EBS snapshot

> **Tip:** Automate cross-Region copies using AWS Lambda and EventBridge to continuously maintain geographically redundant backups as part of DR strategy.

Automation strategies using Data Lifecycle Manager (DLM)

AWS Data Lifecycle Manager (DLM) is a native tool designed to help automate the creation, retention, and deletion of Amazon EBS snapshots. Instead of relying on manual intervention or custom scripts, DLM lets us define policies that handle these tasks in a scheduled and consistent manner. This is particularly beneficial for ensuring compliance with internal data retention policies or external regulatory requirements.

Ask

Snapshots using DLM

DLM works by associating lifecycle policies with EBS volumes using resource tags. These policies specify:

- When to create snapshots (e.g., hourly, daily).
- How long to retain them (e.g., last 7 days).
- Whether to copy them across regions.

Additionally, DLM supports cross-account and cross-Region snapshot management, enhancing both security and resilience.

In contrast to AWS Backup, which provides a centralized service for managing backups across multiple AWS resources, DLM focuses specifically on EBS volumes and AMIs, making it lightweight and easier to configure for simpler snapshot use cases. Here's how we can create DLM using AWS CLI.

main.sh Shell

```
aws dlm create-lifecycle-policy \
  --execution-role-arn arn:aws:iam::<valid_account_id>:role/AWSDataLi
  --description "Daily snapshots for prod volumes" \
  --state ENABLED \
  --policy-details 'Version=1,ResourceTypes=["VOLUME"],TargetTags=[{K
```

Creating a snapshot lifecycle policy with AWS CLI

Ask

> **Tip:** Use meaningful tags like `Backup=true` or `Environment=Prod` to apply DLM policies selectively and consistently.

DLM ensures that EBS data is regularly backed up, even without direct oversight, while keeping costs in check by pruning old, unused snapshots automatically.

Managing snapshots

Once snapshots are created, either manually or through automation, they need to be managed efficiently to avoid unnecessary costs and maintain a clean backup strategy. AWS provides several features that make snapshot management straightforward and scalable, particularly through the use of tagging, filtering, and access controls.

- Tags are key-value pairs used to organize and track snapshots across environments. Tags like `Project=CustomerPortal` or `Lifecycle=Archived` improve discoverability and help with billing, automation, and resource grouping in the AWS Console and CLI.
- Snapshot sharing allows snapshots to be shared across AWS accounts for purposes like migration and recovery. Unencrypted snapshots can be shared directly, while encrypted ones require a shared KMS key with the correct permissions.

Ask

Storing and sharing across accounts

Over time, old or unused snapshots may accumulate, resulting in increased storage costs. Although snapshots are incremental, older snapshots still contribute to data's history and storage footprint. Regular pruning of outdated snapshots is essential. We can do this manually or use scripting with the AWS CLI or SDKs to delete snapshots older than a certain number of days based on their `StartTime` or tag metadata.

We can use the following command to delete a specific snapshot :

main.sh Shell

```
aws ec2 delete-snapshot --snapshot-id <valid_snapshot_ID>
```

Ask

For larger environments or teams, we can set IAM policies to restrict who can create, delete, or modify snapshots. This ensures better governance and reduces the risk of unintended data loss. We can also use AWS Organizations service control policies (SCPs) for broader control over snapshot-related operations at the organizational level.

Monitoring snapshot activity

Monitoring snapshot activity is essential to ensure that backups are successfully created, completed on time, and compliant with internal or external policies. It also helps in identifying operational failures related to snapshot delays, access issues, or missed schedules.

AWS offers several integrated services that support real-time and historical visibility into snapshot workflows. Amazon CloudWatch Events (now integrated into Amazon EventBridge) can be configured to identify when snapshots are started or completed. By pairing EventBridge rules with Amazon SNS, we can send email or SMS notifications to operations teams for real-time alerts. These are especially helpful when working with automated snapshot strategies such as Data Lifecycle Manager (DLM), where visibility into failures is critical.

Ask

Monitoring snapshot

For audit trails, AWS CloudTrail logs every API interaction, including calls to `create-snapshot`, `delete-snapshot`, and `copy-snapshot`. These logs are crucial for compliance and security reviews and can be routed into centralized log processing pipelines for analysis. CloudTrail can also be used to detect unusual activity, such as manual deletions of important snapshots or unauthorized access to volume backups.

Ask

The following EventBridge rule is an example of how we can set up automatic notifications when a snapshot creation event occurs:

main.sh Shell

```
{
  "source": ["aws.ec2"],
  "detail-type": ["EBS Snapshot Notification"],
  "detail": {"event": ["createSnapshot"], "result": ["success"]}
}
```

EventBridge rule to create snapshot

Restoring volumes from snapshots

To restore a volume, we use either the AWS CLI, SDKs, or the Console. From the CLI, the process involves creating a volume from a snapshot and specifying the desired volume type, size (optional), and availability zone. It is important to match the AZ of the new volume with the EC2 instance that will attach to it. Here's the AWS CLI command to restore a volume from a snapshot:

main.sh Shell

```
aws ec2 create-volume \
  --snapshot-id <valid_snapshot_ID>\
  --availability-zone us-east-1a \
  --volume-type gp3
```

Ask

Create volume from snapshot

This command will provision a volume from the specified snapshot in the designated AZ using the `gp3` volume type. Once the volume is created, it can be attached to an EC2 instance using the `attach-volume` API or command.

After attaching the volume, depending on the file system type, we may need to mount the volume manually. On Linux, this typically involves using `mount` commands, and on Windows, we use disk management to bring the volume online and assign a drive letter.

Snapshots for disaster recovery

In more serious scenarios, such as an AWS Region becoming unavailable, snapshots can play a key role in enabling failover. Because snapshots are regional by default, we must explicitly copy them to other regions to make them accessible for recovery.

We can automate cross-Region copy with EventBridge and Lambda. After a snapshot is created, EventBridge triggers a Lambda function to replicate the snapshot:

main.py Python

```python
import boto3

def lambda_handler(event, context):
    source_snapshot = event['detail']['snapshot_id']
    ec2 = boto3.client('ec2', region_name='us-west-2')
    ec2.copy_snapshot(
        SourceRegion='us-east-1',
        SourceSnapshotId=source_snapshot,
        Description='Auto cross-region copy'
```

Ask

```
)
```

Lambda to automate snapshot replication

The flow of snapshot creation using Lambda and EventBridge is as follows:

Storing snapshots across regions

These replicated snapshots can be used to launch new EC2 instances and restore services in the backup region. This pattern is essential for businesses with low Recovery Time Objectives (RTOs) and regulatory obligations for geographic redundancy.

Let’s consider a scenario where snapshots in CI/CD pipelines can be useful in case of failures.

Ask

Snapshots in CI/CD pipelines

In modern CI/CD workflows, especially when dealing with stateful applications, protecting persistent data is critical. Amazon EBS snapshots provide an effective way to safeguard that data by allowing us to capture the state of a volume before each deployment.

Imagine a scenario where an EC2 instance is running our application, and that instance relies on an attached EBS volume to store essential application data. With each new build pushed through the CI/CD pipeline, more data is written to the volume, data that must not be lost, even if a deployment fails.

To address this, the pipeline can be configured to create a snapshot of the EBS volume before deploying a new version of the application. If something goes wrong during deployment, such as the application becoming unstable or the instance becoming inaccessible, the snapshot acts as a recovery point. We can launch a new EC2 instance, restore the EBS volume from the most recent snapshot, and attach it to the new instance. This allows us to revert the environment to a previously functional state quickly and continue operations with minimal disruption.

By integrating snapshot creation into the deployment process, we ensure that our application's critical data remains safe and recoverable, even during unexpected failures. The illustration below shows how the recovery works:

Ask

Ask